COMSATS University Islamabad (Lahore Campus)

Department of Computer Engineering

Implementation Guide

Member A — Platform, MQTT & Deployment

Detailed Sprint Breakdown & Approach

Companion: `member_a_sprint_plan.tex` · Master: `centralized_sprint_plan.tex`

Spring–Fall 2026

Contents

Chapter 1

Introduction

1.1 Purpose

This guide tells you **how to implement** your track week by week — not just *what* to build. Use it alongside:

- `.sprint/member_a_sprint_plan.tex` — task list
- `.sprint/centralized_sprint_plan.tex` — joint dates & milestones
- `.prerequisite/MEMBER_A_PREREQUISITES.md` — skills before Sprint 1
- `finalization/dinosaur_fyp_architecture_92471e72.plan.md` — full architecture

1.2 Your boundary

You own everything from MQTT publish through to the dashboard. Member B stops at a Python handoff dict. **Never wait for Jetson until Sprint 4** — the event simulator is your edge substitute in Sprints 1–3.

1.3 General workflow every week

1. **Monday:** Pick Jira tickets for the week from active sprint
2. **Daily:** Code on feature branch; push at least every 2 days
3. **Wednesday:** 30-min sync with Member B (blockers only)
4. **Friday:** Open/update PR; comment on Jira ticket
5. **Sunday:** Update `docs/sprint-logs/` weekly entry

Tip

Build **vertically thin slices** each sprint: make one event flow end-to-end before adding features. Example Sprint 2: one simulator alert → DB row → REST query — before polish.

Chapter 2

Sprint 1 (Weeks 1–4): Foundation & Transport

2.1 Sprint goal

Monorepo exists, contract drafted, Mosquitto + Redis + bridge + simulator running. Member B aligns handoff spec. **Milestone M0** at week 4.

How to Approach This Sprint

Order of work:

1. Dev environment + GitHub/Jira setup (Day 1–2)
2. Repo scaffold + `contracts/events.py` (Week 1)
3. Docker Compose with Mosquitto + Redis only (Week 2)
4. Bridge + simulator (Week 3)
5. Contract sign-off with B + integration session (Week 4)

Do not start FastAPI services yet — transport must work first.

2.1.1 Week 1–2: Scaffold & contract

Step: Initialize monorepo

Create top-level layout:

```
smart-surveillance/
|-- contracts/events.py
|-- transport/mqtt_redis_bridge/
|-- transport/event_simulator/
|-- edge/mqtt_publisher/        # you own; B never commits here
|-- services/alert_service/
|-- infra/docker-compose.yml
|-- frontend/src/
```

Open Jira project SSP; create Sprint 1; tickets: SSP-1 scaffold, SSP-2 contract, SSP-3

compose.

Step: Draft frozen contract

In `contracts/events.py` define Pydantic models:

- `type`: alert — metrics — heartbeat
- `event`: helmet_missing, vest_missing, fire, smoke, spill, compliant
- Fields: timestamp, session_id, worker_id, zone, confidence, compliance_score, detections[], image_path

Share PDF/markdown mapping table with Member B: handoff dict field → contract field.

Step: Docker Compose skeleton

Start minimal `infra/docker-compose.yml`:

- Network `ppe-network`
- Services: `mosquitto`, `redis` (health checks)
- Volumes for Redis persistence
- `.env.example` with `MQTT_HOST`, `REDIS_URL`

Verify: `docker compose config` passes.

2.1.2 Week 3–4: Bridge & simulator

Step: MQTT-Redis bridge

Implement async Python service:

1. Subscribe to `ppe/#` on Mosquitto
2. Parse JSON; validate with `contracts.events`
3. Route: `ppe/alerts/*` → `XADD ppe:alerts`
4. Add metadata: `received_at`, `device_id`
5. On parse fail: `XADD ppe:dead_letter`
6. Exponential backoff reconnect

Test: `mosquitto_pub` manually → `XRANGE ppe:alerts`.

Step: Event simulator

Publish realistic events at 45/hour:

- All zones: welding_bay, general, chemical_storage
- All priorities; include compliant + violation mix
- Valid `image_path` strings (no real files needed yet)

This replaces Jetson until Sprint 4.

Verification Checklist

- ☐ `docker compose up` starts mosquitto, redis, bridge, simulator
- ☐ `XRANGE ppe:alerts - + COUNT 10` shows events
- ☐ Bridge survives restart (Redis persistence)
- ☐ Member B signed handoff mapping doc (M0)
- ☐ First PR merged; Jira Sprint 1 tickets closed

Collaboration Touchpoint

Week 4 integration: Walk Member B through contract fields. Receive sample handoff JSON from B; write a one-off script that converts it to contract payload (preview for Sprint 4 publisher).

Common Pitfalls

- Starting microservices before bridge works — debug transport in isolation first
- Changing contract after M0 without B's approval
- Committing `.env` secrets to GitHub

Chapter 3

Sprint 2 (Weeks 5–8): Backend Core

3.1 Sprint goal

PostgreSQL + MinIO live; Alert and Metrics services consume streams and expose REST. **Milestone M1** at week 8.

How to Approach This Sprint

Order of work:

1. Database schema + migrations (Week 5)
2. Alert Service consumer only — log to console (Week 5)
3. Alert Service DB write + dedup (Week 6)
4. Metrics Service + hypertable (Week 7)
5. REST endpoints + integration test (Week 8)

Keep simulator running entire sprint.

3.1.1 Week 5–6: Database & Alert Service

Step: PostgreSQL schema

Add to Compose: PostgreSQL + TimescaleDB image.

```
violations (id, timestamp, worker_id, zone,
  event_type, priority, confidence, image_path,
  agent_briefing, acknowledged)

compliance_metrics (time, zone, shift, score,
  worker_count, violation_count)  -- hypertable

zone_rules (zone, requires_helmet, requires_vest)
```

Run migrations; seed `zone_rules` for 3 zones.

Step: MinIO

Add MinIO service; create `evidence/` bucket. Alert service will store/fetch paths — upload test JPEG manually to verify presigned URLs.

Step: Alert Service (:8001)

1. Consumer group `alert-svc` on `ppe:alerts`
2. Dedup: Redis key `dedup:{worker}:{event}:{zone}` TTL 60s
3. INSERT into `violations`; `XACK` after success
4. CRITICAL priority: skip rate limit
5. Publish to Redis pub/sub channel `live:alerts` for Sprint 3 WebSocket
6. `GET /api/alerts` with filters

3.1.2 Week 7–8: Metrics Service

Step: Metrics Service (:8002)

1. Consumer `metrics-svc` on `ppe:metrics`
2. Write to `compliance_metrics` hypertable
3. Rolling hourly/daily aggregates per zone
4. Zone risk score: weighted violations × severity
5. Endpoints: `/api/metrics/compliance`, `/trend`, `/zone-risk`

Step: Integration test

Pytest with docker-compose:

1. Start stack + simulator
2. Wait 30s; assert `violations` count > 0
3. `GET /api/alerts` returns 200
4. `GET /api/metrics/compliance` returns data

Verification Checklist

- ☐ M1: simulator → Redis → Alert → PostgreSQL
- ☐ REST returns filtered alerts
- ☐ Metrics hypertable has rows
- ☐ Ingested B's sample handoff JSON via test script (optional)

Collaboration Touchpoint

Week 8: Integration session. Demo REST API to Member B. Collect any handoff JSON samples for Sprint 3 harness.

Chapter 4

Sprint 3 (Weeks 9–12): Real-Time, Reports, Synthetic, Agent

4.1 Sprint goal

WebSocket live; 90-day synthetic seed; PDF reports; agent skeleton with 3 tools. **M2** at week 12.

How to Approach This Sprint

Parallel tracks within sprint:

- **Track A:** WebSocket + Reporting (Week 9–10) — demo value fast
- **Track B:** Synthetic data (Week 9–10) — unblocks dashboard/agent testing
- **Track C:** Agent skeleton (Week 11–12) — after DB has synthetic history

Review Member B's merged PRs in `edge/` but do not write CV code.

4.1.1 Week 9–10: WebSocket & Reporting

Step: WebSocket Gateway (:8004)

1. Subscribe Redis `live:alerts` and metrics channels
2. FastAPI `/ws/live`; connection manager (add/remove clients)
3. Broadcast JSON on each alert
4. Test with `websocat` or browser DevTools

Step: Reporting Service (:8003)

ReportLab PDF: header, date range, compliance summary, violation table, recommendations paragraph. `POST /api/reports/generate`. Scheduled cron optional stub.

Step: Synthetic data engine

`synthetic/scenarios/90day_baseline.yaml`: 15 workers, 3 zones, patterns (night drop, welding bay chronic violations). Run `python synthetic/generate.py` — bulk INSERT + optional stream replay. Dashboard and agent now have history.

4.1.2 Week 11–12: Agent skeleton

Step: Agent Service (:8005)

1. LangGraph: planner → tool executor → responder
2. Tools calling your DB: `query_violations`, `get_compliance_trend`, `get_zone_risk_score`
3. `POST /api/agent/query` with OpenAI GPT-4o; Ollama fallback env flag
4. OpenAPI on all 5 services

Step: Handoff test harness

Script: read B's handoff JSON file → your publisher function → MQTT → verify row in DB. No Jetson required.

Verification Checklist

- ☐ WebSocket client receives event within 2s of simulator publish
- ☐ PDF downloads with synthetic data charts
- ☐ Agent answers: "How many violations in welding bay last 24h?"
- ☐ B's edge PRs merged; harness passes with B's sample dict

Chapter 5

Sprint 4 (Weeks 13–16): Dashboard & Jetson

5.1 Sprint goal

Full React UI; MQTT publisher on Jetson wrapping B's code; agent RAG + reactive mode. **M3** at week 16.

How to Approach This Sprint

Order of work:

1. React shell + WebSocket + Dashboard page (Week 13) — use synthetic data immediately
2. Remaining pages (Week 14)
3. `edge/mqtt_publisher/` adapter for B's handoff (Week 15)
4. Jetson flash + deploy + disable simulator (Week 15–16)
5. Agent RAG + reactive CRITICAL briefings (Week 16)

5.1.1 Week 13–14: React dashboard

Step: Frontend scaffold

Vite + React + TypeScript + Zustand + Tailwind. WebSocket hook updates store. Pages:

- Dashboard: compliance gauge, mini alert feed
- Alerts: full feed, filters, evidence modal
- Analytics: trend chart, zone heatmap, pie chart
- Agent: chat panel
- Reports: date picker → PDF
- Config: helmet/vest toggles per zone

5.1.2 Week 15–16: Jetson integration

Step: MQTT publisher on edge

`edge/mqtt_publisher/publisher.py`:

1. Import B's `get_compliance_event(frame)` from compliance engine
2. Map dict → `contracts.events.AlertPayload`
3. Add timestamp, session_id, priority, type
4. paho-mqtt publish to broker (broker IP = your laptop/server on network)

Step: Jetson deploy

1. Flash JetPack; SSH; install deps from B's `requirements.txt`
2. Copy B's weights/ONNX; verify inference FPS
3. Run publisher + inference loop as systemd service
4. Point Jetson MQTT at Compose Mosquitto IP
5. Verify end-to-end; stop simulator

Step: Agent completion

pgvector embeddings on violations; reactive trigger on CRITICAL stream event; write `agent_briefing` to DB; push via WebSocket.

Verification Checklist

- ☐ M3: remove helmet on camera → gauge drops → alert on UI
- ☐ Agent briefing appears on CRITICAL alert
- ☐ All 6 pages functional

Collaboration Touchpoint

Weeks 15–16: B available for threshold tuning questions. You operate Jetson during integration session.

Chapter 6

Sprint 5 (Weeks 17–20): DevOps & Demo

6.1 Sprint goal

One-command stack; CI/CD; Grafana; 12-minute demo rehearsed. **M4** at week 20.

How to Approach This Sprint

Do DevOps **after** features work — don't block Sprint 4 UI on Grafana. Week 17–18: Compose hardening + CI. Week 19–20: demo script + report.

Step: Full docker-compose

All services with `depends_on` health conditions. Nginx serves React build. Document `README.md` quick start.

Step: GitHub Actions

Lint (ruff, eslint) → pytest → compose integration job → build images.

Step: Observability

Prometheus scrape all `/metrics`. Grafana dashboards: System Health + Safety KPIs. Show both during demo.

Step: 12-minute demo script

1. `docker compose up` (1 min)
2. Grafana health (1 min)
3. Live violation + agent briefing (2 min)
4. Agent chat question (2 min)
5. Analytics 90-day view (2 min)
6. PDF report (1 min)

7. Zone config change (1 min)
8. Q&A architecture (2 min)

Rehearse twice; record screen backup.

Verification Checklist

- ☐ M4: demo run without manual fixes
- ☐ CI green on `main`
- ☐ Platform report chapters drafted
- ☐ Tag `v1.0-demo`

Chapter 7

Appendix A: Service Ports

Port	Service
1883	Mosquitto
6379	Redis
5432	PostgreSQL
9000	MinIO
8001–8005	FastAPI services
3000	React (Nginx)
9090 / 3001	Prometheus / Grafana

Chapter 8

Appendix B: Troubleshooting

- **No stream messages:** Check bridge logs; `XRANGE`; Mosquitto ACL
- **Consumer lag:** `XPENDING ppe:alerts alert-svc`; claim stale messages
- **WebSocket silent:** Verify Redis pub/sub from Alert Service
- **Jetson cannot reach broker:** Same LAN IP; firewall port 1883
- **Agent empty answers:** Check synthetic seed; OpenAI key; tool SQL